

Trustless Delegation of Vector Commitment Construction in Resource-Constrained Settings

Parisa Hassanizadeh
IPPT PAN / Zero Savvy
parisa@zerosavvy.xyz

Stefan Dziembowski
University of Warsaw
stefan.dziembowski@crypto.edu.pl

Shahriar Ebrahimi
Zero Savvy
shahriar@zerosavvy.xyz

Janusz Szczepanski
IPPT PAN
jszczepa@ippt.pan.pl

ABSTRACT

Many data types, such as video and audio, consist of sequential elements where both integrity and order are essential for authenticity. In practice, verifiers often access only partial sequences due to privacy or volume constraints, as in surveillance, journalism, and public records. Vector commitment (VC) schemes that support verifiable partial disclosures address this need, but constructing and maintaining such VCs is challenging for resource-constrained devices such as cameras. For example, in CCTV systems, a trusted module updating the VC for millions of frames must store and process hundreds of megabytes, which is often infeasible.

We propose a proving system that enables trustless reconstruction of VCs from only the chained hash of the sequence. The data source computes and signs a cumulative hash. An untrusted prover then constructs the VC from the raw data and proves that the data used for construction is consistent with the signed hash chain. The constructed VC subsequently enables authentication of partial disclosures. The main challenge is that proving the full VC construction in zero-knowledge is computationally expensive. To address this, we design a folding-based zkSNARK tailored to this task.

We analyze the construction in a realistic setting with a constrained source device (Raspberry Pi Zero) and a consumer-grade prover (midrange laptop). Our results show that even for relatively short sequences (e.g., 30 minutes of video footage), handling the VC within the source device’s trusted module is infeasible due to both storage and computational limitations. In contrast, the proposed trustless offloading imposes only a few minutes of local computation on the prover to generate a proof for full VC construction (around 2 minutes for the 30 minutes of footage). Our implementation is available as open source at: <https://github.com/zero-savvy/proven-view>.

1 INTRODUCTION

Modern technologies and applications increasingly generate or consume big data that can be structured sequentially, where integrity depends not only on the content of individual elements but also on their order. Preserving this order is essential for various data types, including media files, financial records, sensor outputs, and health monitoring data. Ensuring the authenticity and integrity of such data is critical for accurate decision-making, logical reasoning, and system behavior analysis. Meanwhile, the emergence of generative AI and deepfakes has intensified concerns around misinformation, including fraud, phishing, and identity forgery, by enabling the low-cost creation of synthetic content. As a result, the authenticity

of diverse data types, in particular video and audio recordings, has become an increasing concern in both academia and industry.

Authenticating these sequential data requires a reliable binding to their original source, ideally generated by a trusted entity such as a tamper-proof module in a camera [1, 3, 29]. Existing industry methods primarily rely on digital signatures, which can effectively certify the authenticity of entire sequence [28]; However, this means that in order to verify, the verifier would always need to have access to the entire source. On the other hand, recordings often contain extra content unrelated to the core event, presenting a significant challenge: “*Authenticating and verifying arbitrary trims without revealing the rest of the original sequence.*” This is particularly critical in scenarios where privacy concerns restrict full disclosure of the source, such as court proceedings, investigative journalism, and whistleblower cases.

Some studies propose AI-based detection systems as a solution aiming at verifying the authenticity without access to the original source [26, 34, 40]. However, these systems often struggle with generalization and tend to perform poorly across diverse generative methods and video transformations [32]. In parallel, initiatives such as the C2PA¹ advocate for embedding metadata into media to certify its origin and editing history [5]. This metadata is validated through digital signatures issued by trusted entities, such as applications executing within Trusted Execution Environments (TEEs) [5]. Companies including OpenAI and Adobe have adopted C2PA in products such as Photoshop and DALL-E 3 [4]. However, TEEs introduce additional trust concerns, restrict verification to specific hardware platforms, and expose the system to new attack vectors [12, 23].

Another growing concern with frameworks like C2PA is the *concentration of authority over individual data*, posing risks in settings such as anonymous journalism and whistleblowing. To address this, a parallel research direction explores using zero-knowledge proofs (ZKPs) to certify editorial modifications without revealing the original source [13, 14, 41]. These approaches require a binding commitment to link the proof to the source, for which recent works suggest optimized vector commitment (VC) schemes [33, 38]. Such schemes, often based on Merkle trees, support verifiable partial disclosure of elements from a private source. Nonetheless, practical challenges remain. First, archival formats such as CCTV footage introduce variability in frame sizes and encoding schemes, complicating frame-level integrity and temporal consistency. Second, most prior work neglects the computational overhead of commitment

¹Coalition for Content Provenance and Authenticity

construction; in real-world settings, such as live video capture, devices already perform intensive image processing, making real-time maintenance of VC structures difficult, and in many scenarios, as shown in our evaluation (Section 7), infeasible.

To address this problem, we propose a protocol that allows trustlessly proving the correct delegated construction of a VC from only the chained hash of the sequential data. This approach enables the original data creator (such as a capturing device) to compute and sign the hash of data as usual, without incurring additional computational overhead. Subsequently, the prover generates a proof that they have constructed a VC correctly w.r.t. the signed hash of the original data. This proof serves as an authentication certificate, trustlessly binding the VC to the original hash. Once established, the VC can now be used to efficiently prove the authenticity of partial disclosures. We implemented the full protocol as an open-source library and formally prove its security under the probabilistic polynomial-time (PPT) adversary model. We evaluate it in a realistic setting where the original data creator has constrained resources (e.g., a camera), and the prover operates on a mid-range consumer-grade laptop. The key contributions of this paper are as follows:

- **Trustless and publicly verifiable efficient construction of VC.** We propose a method for (re)constructing VC derived solely from the *chained hash* of sequence elements. The proposed method is format-agnostic, allowing it to iterate over elements of varying sizes and to be efficiently proven in a folding-based scheme. To the best of our knowledge, this is the first approach that enables verifiable reconstruction of a VC within a ZKP, leveraging folding schemes.
- **Authenticity of arbitrary trims.** Our proof system support scalable and efficient verification of trimmed segments. The proof complexity is not affected by the trim size or its location. We formally analyze the *soundness* of our constructions.
- **Efficient and practical implementation.** We have implemented the entire protocol as an open-source library on GitHub². We evaluate its performance in a realistic setting, where a mid-range laptop acts as the *prover* and a *Raspberry Pi* board serves as the source of the original sequence.

The remainder of the paper is organized as follows. Section 2 reviews the necessary background. Section 3 surveys related work, outlines target applications, and describes the system and adversary model. Section 4 presents our VC scheme and its trustless construction based on a chained hash. Section 5 shows how our VC enables efficient authenticity proofs for partial disclosures. Section 6 details the protocol implementation and circuit design. Section 7 provides our experimental results. Finally, Section 8 concludes the paper.

2 BACKGROUND

In this section, we introduce the fundamental principles of VC schemes and non-interactive proving systems, with a particular emphasis on SNARKs. Our incrementally verifiable computation (IVC) constructions adopt the notation and methodology from [24, 25].

²<https://github.com/zero-savvy/proven-view>

Table 1: TPM Storage Limitations

Feature	Infineon SLB 9670	ST ST33T-PHF2ESPI	Nuvoton NPCT750	Intel PTT
NV Storage	64 KB	34 KB	58 KB	Emulated
Max Cmd	4096 B	4096 B	4096 B	4096 B
Max Resp.	5120 B	5120 B	5120 B	5120 B
NV Buf.	2048 B	1024 B	2048 B	1024–2048 B
Trans. Obj.	5	3–5	~5	N/A
ECC	Yes	Yes	Yes	Yes

We provide a summary of the key symbols and terminology used throughout the paper in Table 2.

2.1 Trustworthy and Tamper-Proof Camera Systems

Modern approaches to trustworthy video capture increasingly rely on tamper-proof hardware components such as Trusted Platform Modules (TPMs) to establish root-of-trust. It is worth noting that TPMs are primarily deployed in attestation protocols rather than in application-level cryptographic primitives [15]. These modules are designed to perform a limited set of cryptographic operations securely, and their use in constrained environments like embedded cameras introduces fundamental limitations in terms of computation, storage, and throughput. Table 1 provides a detailed comparison of the most widely used TPM chip architectures in industry.

TPMs are equipped with volatile (RAM) and non-volatile (NV) memory. The internal RAM is typically used for transient keys, session data, and temporary buffers during operations like hashing or signing. However, this memory is not directly accessible to users and is severely limited. For example, the number of transient objects that can be loaded simultaneously ranges between 3 and 10, depending on the vendor. Non-volatile storage is also highly constrained, typically ranging from 32 to 64 KB. These limits make it impractical to assume such modules could potentially be employed by application-level cryptographic needs in the source device for example to maintain large cryptographic data structures such as VCs.

To mitigate these constraints, TPM software stacks (TSS) implement several techniques such as context swapping, NV storage emulation, and policy digest offloading. For instance, large inputs (e.g., video segments or measurement logs) must be processed in a streaming fashion using sequence-based operations like `TPM2_HashSequenceStart` and `TPM2_SequenceUpdate`. Ultimately, only compact outputs—such as cumulative digests or policy digests—are kept within the TPM. This architectural design aligns with our assumption that trustworthy camera modules can, at best, compute and sign a chained hash over sequential data, rather than maintaining full cryptographic state or structures like VCs internally.

Recent academic efforts have also investigated hardware-based secure video pipelines. ProxCam [29] is a self-contained camera module that is capable of authenticating video footage during capture. ProxCam implements a custom TCB on an FPGA and chains

video frames using cumulative hashing. Its design emphasizes minimizing tampering throughout the hardware-software pipeline, achieving greater efficiency in application-specific scenarios compared to TPM-based solutions. These findings reinforce our assumption that trustworthy camera systems are inherently constrained to producing chained hashes, rather than maintaining full vector commitments, which is supported by both industry practices and the current academic literature.

2.2 Vector Commitment

A VC is a cryptographic primitive that functions as an accumulator, enabling a committer to bind to an ordered sequence of values. This allows for selective openings of the commitment at specific positions, providing compact proofs. The VC scheme offers public verifiability, meaning a verifier can confirm the authenticity of individual elements within the committed vector without the need for having access to the entire sequence [10, 11, 38].

Definition 2.1. A VC scheme can be defined as a quadruple of PPT algorithms $VC = (\text{KeyGen}_\Delta, \text{Com}_\Delta, \text{Open}_\Delta, \text{Vfy}_\Delta)$ as follows:

- $pp \leftarrow \text{KeyGen}_\Delta(1^\lambda, n)$: The generator outputs public parameters pp based on the security parameter λ , and message space size of n .
- $\Delta_M \leftarrow \text{Com}_\Delta(pp, m_1, \dots, m_n)$: Commits to all the elements in the sequence $\{m_1, \dots, m_n\}$.
- $\delta_k \leftarrow \text{Open}_\Delta(pp, m_k, k, \text{aux})$: Produces a proof for opening the commitment corresponding to the k^{th} element in the sequence, where $k \in [n]$. The opening usually requires knowledge of some auxiliary value aux .
- $b \leftarrow \text{Vfy}_\Delta(pp, \Delta_M, m_k, k, \delta_k)$: The verifier accepts the proof δ_k iff m_k is indeed the k^{th} value of the accumulated Δ_M .

VC schemes can also support advanced features such as updatability and batch proofs [10, 38]. However, for simplicity, we focus on the basic definition while noting that these features remain compatible with our constructions. A VC scheme satisfies **position-binding** if, for every index i in the range $[n]$ and for any PPT adversary $\mathcal{A}$, the probability that $\mathcal{A}$ successfully opens the commitment at index i with two different values is negligible in the security parameter λ [11].

2.3 Non-Interactive Zero-Knowledge Arguments of Knowledge

We follow the original definition from [21] to formally define Non-Interactive Zero-Knowledge Arguments of Knowledge.

Definition 2.2. Let $\mathcal{R}$ be a relation generator that, given a security parameter λ in unary, outputs a binary relation R that is decidable in polynomial time. For any pair $(\phi, w) \in R$, we refer to ϕ as the *statement* and w as the *witness*. Let $\mathcal{R}_\lambda$ denote the set of relations that $\mathcal{R}$ may output when given input 1^λ . Now, we define a publicly verifiable non-interactive argument system for $\mathcal{R}$ as a quadruple of probabilistic polynomial algorithms $\Pi = (\text{Setup}, \text{Prv}, \text{Vfy}, \text{Sim})$ such that:

- $(\sigma, \tau) \leftarrow \text{Setup}(R)$: produces the CRS σ and a trapdoor τ for the relation R .

Table 2: Terminology of the Paper

Notation	Description
$X = \bigcup x_i$	A sequence of n elements, where $\forall i : x_i = s_i \times \lfloor \log p \rfloor$, s.t. $x_i \in \mathbb{Z}_p^{s_i}$.
H	A collision-resistant block hash, such as Poseidon [20]. $H : \mathbb{Z}_p^2 \rightarrow \mathbb{Z}_p$
H_ϕ	Array/frame hasher (§4.1 Def. 4.1). $H_\phi : \mathbb{Z}_p^S \rightarrow \mathbb{Z}_p$
H_Φ	Proposed sequence hasher (§4.1 Def. 4.1). $H_\Phi = H(\ell_x^{i-1} \ H_\phi(x_i))$.
ℓ_x^i	Cumulative hash of the original sequence until the i^{th} element.
σ_m	Digital signature of message m .
Δ_M	Root of the Merkle tree built over the vector $M = \{m_1, \dots, m_n\}$.
δ_i	Merkle proof of the i^{th} leaf.
π	zkSNARK proof: we use notation such as π_τ , π_σ , and π_Δ .
Vfy	Generally, denotes the verification algorithm. We specifically use Vfy_Δ , Vfy_σ and Vfy_π for commitment, signature and zkSNARKs, respectively.

- $\pi \leftarrow \text{Prv}(R, \sigma, \phi, w)$: given that $(\phi, w) \in R$, the algorithm returns an argument π .
- $b \leftarrow \text{Vfy}(R, \sigma, \phi, \pi)$: using the same statement ϕ and CRS σ used in generating π , the algorithm Vfy returns $b \in \mathbb{Z}_2$: 0 (reject) or 1 (accept).
- $\pi \leftarrow \text{Sim}(R, \tau, \phi)$: Given the trapdoor τ and statement ϕ , the simulator can return the argument π .

Definition 2.3. According to [21], we can label Π as a “*perfect non-interactive zero-knowledge argument of knowledge*” if it satisfies following properties: **Perfect Completeness**: An honest prover with valid witness should convince an honest verifier. Formally:

$$\Pr \left[\text{Vfy}(R, \sigma, \phi, \pi) = 1 \mid \begin{array}{l} (\sigma, \tau) \leftarrow \text{Setup}(R) \\ (\phi, w) \in R \\ \pi \leftarrow \text{Prv}(R, \sigma, \phi, w) \end{array} \right] = 1$$

Definition 2.4. Computational Soundness: The Π is sound if it is not possible to prove a false statement. Let L_R denote the language consisting of all statements such as ϕ that have matching witness in R . Therefore, for all PPT adversaries $\mathcal{A}$:

$$\Pr \left[\begin{array}{l} \text{Vfy}(R, \sigma, \phi, \pi) = 1, \\ \phi \notin L_R \end{array} \mid \begin{array}{l} (R, z) \leftarrow \mathcal{R}(1^\lambda) \\ (\sigma, \tau) \leftarrow \text{Setup}(R) \\ (\phi, \pi) \leftarrow \mathcal{A}(R, z, \sigma) \end{array} \right] = \text{negl}(\lambda)$$

Computational Knowledge Soundness: To call Π an “*argument of knowledge*”, we require existence of a PPT extractor $\mathcal{X}_\mathcal{A}$ for all PPT adversaries $\mathcal{A}$ that can compute the witness whenever the adversary manages to provide a valid argument:

$$\Pr \left[\begin{array}{l} \text{Vfy}(R, \sigma, \phi, \pi) = 1, \\ (\phi, w) \notin R \end{array} \mid \begin{array}{l} (R, z) \leftarrow \mathcal{R}(1^\lambda) \\ (\sigma, \tau) \leftarrow \text{Setup}(R) \\ ((\phi, \pi), w) \leftarrow (\mathcal{X}_\mathcal{A} \| \mathcal{A})(R, z, \sigma) \end{array} \right] = \text{negl}(\lambda)$$

Definition 2.5. Perfect Zero-knowledge: The argument does not leak anything beyond the truth of the statement. We say Π

is perfect zero-knowledge if for all adversaries $\mathcal{A}$ with access to z , which is an auxiliary side information derived from relation generator:

$$\Pr \left[\begin{array}{c} \mathcal{A}(R, z, \sigma, \\ \tau, \pi) = 1 \end{array} \middle| \begin{array}{c} (\sigma, \tau) \leftarrow \text{Setup}(R) \\ (\phi, w) \in R \\ \pi \leftarrow \text{Prv}(R, \sigma, \phi, w) \end{array} \right] = \Pr \left[\begin{array}{c} \mathcal{A}(R, z, \sigma, \\ \tau, \pi) = 1 \end{array} \middle| \begin{array}{c} (\sigma, \tau) \leftarrow \text{Setup}(R) \\ (\phi, w) \in R \\ \pi \leftarrow \text{Sim}(R, \tau, \phi) \end{array} \right]$$

Incrementally Verifiable Computation (IVC). Folding-based zkSNARKs are a set of specific cryptographic primitives that achieve IVC by allowing verification of computations done by repeated application of the same function. More precisely, for a given function f , with initial input z_0 , an IVC scheme generates a proof Π_i for stating that $z_i = f(z_{i-1}) = f^i(z_0)$, given a proof Π_{i-1} for stating $z_{i-1} = f^{i-1}(z_0)$. An interesting property of IVC schemes in general is that they support additional auxiliary inputs for f . While the main input for each iteration of applying f comes from previous step, the auxiliary input ω_i in each step is independent of other steps. Nova [25] was one of the first to construct a compressed zkSNARK from an IVC scheme. To achieve this, they introduced the concept of folding schemes, which transform the verification of two NP statements into checking a single NP statement. The result was a final compressed Spartan zkSNARK, which is a transparent SNARK, meaning that it does not require any trusted setup [35]. The Nova-snark library [6] provides one of the fastest provers for the Nova protocol. This library supports multiple frontends to define steps in Nova, such as Circom [9] through the nova-scotia library [7].

3 RELATED WORK AND TARGET APPLICATIONS

The most relevant works to ours focus on verifying data originality and adopt a range of approaches, including watermarking, perceptual hashing, cryptographic techniques, and hardware-based solutions. Each category offers distinct advantages and faces specific limitations, particularly in terms of privacy requirements, minimum computational and storage requirements, and overall efficiency. In this section, we first review the strengths and shortcomings of these methods. We then introduce the target application scenarios and system assumptions that motivate the design choices of our approach.

3.1 Related Work

Visual Fingerprinting Methods. Both watermarking and perceptual hashing aim to enable post hoc identification of media content by embedding or deriving resilient signatures. Watermarking embeds invisible markers during generation, with recent methods [16, 17, 27] demonstrating partial robustness to common transformations. However, their reliability decreases under combined edits such as cropping with compression or contrast adjustment [18, 22, 30]. Perceptual hashing, in contrast, computes fingerprints based on visual features, allowing similar outputs for visually similar inputs [36]. Despite its use in real-world systems such as Apple’s CSAM detection [2], perceptual hashing remains sensitive to adversarial changes and may leak content information [37].

Digital Signatures. If the point of media capture is assumed to be trusted (e.g., a TEE or TPM or any other trusted hardware module [29]), digital signature schemes can be effectively used to enable provenance verification of the original raw media by embedding metadata and signing the content [5, 29]. This approach underpins frameworks such as C2PA [5], which has been adopted by major industry players including Adobe, Microsoft, and more recently, OpenAI [4]. However, a key limitation is that signatures apply only to the original media: even simple, non-invasive transformations (such as resizing from 8K to HD) produce outputs without valid signatures. This limitation stems from the requirement that the editor entity is required to be both trusted and tamper-proof. While trusting a hardware module within a camera is already nontrivial, C2PA-based workflows go further by assuming the integrity of the editing environment (e.g., Photoshop running inside a TEE), which introduces additional attack surfaces [39]. Consequently, many industry systems [5, 28, 29] face barriers to widespread adoption, since maintaining a trusted environment is also required during post-processing. This is particularly problematic in domains such as journalism, where edits are not merely aesthetic but often essential for protecting privacy, for example, by redacting individuals at risk.

Zero-Knowledge Proofs. In contrast to C2PA-based approaches, recent research has explored the use of ZKPs to eliminate the need to trust editing software or tools. Systems such as VIMz [14] and VerITAS [13] demonstrate that ZKPs can authenticate edits in images at resolutions up to 8K. Extending these techniques to video, however, introduces new challenges due to the sequential nature of frames. For example, Zhang et al. [41] investigate ZKP-based video authentication for edits such as *redaction* and *blurring* on lossy-encoded videos.

We note that our focus in this paper is orthogonal to prior work, as we target the authenticity of *trims*, a temporal operation typically performed prior to any spatial edits. Specifically, we concentrate on assigning a proper verifiable VC to a sequence that can later be leveraged by methods such as [41]. Without a proper VC to handle trims before editing, approaches like [41] would exhibit computational complexity that scales with the full length of the source video. In contrast, our construction enables subsequent techniques to operate directly on the trimmed segment, achieving complexity that is independent of the original duration.

VC Schemes. Several studies extend the functionality and scalability of VCs beyond simple Merkle tree structures, either by modifying parent node constructions or by introducing new methods for efficient updatability. Pointproofs [19] introduce a VC that supports non-interactive aggregation of proofs across multiple commitments. Reckle [33] presents a VC construction based on succinct recursive arguments combined with Merkle trees, making it particularly suitable for Map/Reduce-style computations. In [38], the authors analyze the trade-off between update information size and proof update time in dynamic VCs, and propose a family of schemes with sublinear update complexity. All of these approaches are compatible with our construction and can be integrated into our framework. In particular, our VC structure can be extended to benefit from the optimizations proposed in [19, 33, 38].

3.2 Target Application and Desired Properties

VCS enable efficient proofs of authenticity for partial disclosures from private data. They find applications across domains such as finance, healthcare, and journalism, where sensitive information must be selectively shared without compromising privacy. Spatial edits such as redaction or blurring can protect privacy, but temporal trimming is typically performed first. This, requires constructing VCs over large media streams. However, a challenge here is that continuous sources such as CCTV recordings generate data that must be incrementally incorporated into the VC. Maintaining such commitments is both computationally and storage intensive, particularly under the limited constraints of real-world devices (see Section 2.1). Moreover, most related work [13, 14, 41] assumes VC construction with ZK-friendly primitives (e.g., Poseidon [20]), which are not yet widely supported in hardware, unlike standardized primitives such as SHA or MD5. Consequently, assuming that commodity trusted devices (e.g., cameras) can construct and maintain fully usable VCs during recording is unrealistic. Our evaluations in Section 7 confirm this limitation in practical scenarios. Prior work [29] similarly observes that even custom secure camera modules struggle to maintain a chained hash at standard video rates, let alone a full VC. This motivates designs that derive VCs from signatures over cumulative hashes of media sequences.

Overall, a practical solution must satisfy the following requirements:

- **Format Agnosticism:** Operate across heterogeneous media formats and compression schemes, including mixed frame types in video streams.
- **Practicality:** Support long sequences with minimal computational and storage overhead at the source device.
- **Efficient Partial Disclosures:** Enable trustless authentication of partial disclosures (e.g., trims) of the original sequence.

3.3 System and Adversary Model

Figure 1 illustrates the system model and its main actors as follows.

- **Source ($\mathcal{S}$).** A device or entity that produces the original sequence and embeds a verifiable final signature in it. This may be an authenticated camera [5, 29] or a certified content creation tool [4]. Examples include Sony’s Alpha 7 cameras in “in-camera signing” mode [3] or the Snapdragon 888 mobile processor providing signing compatible with C2PA [1]. Alternatively, related work such as [29] can also be used as a custom trusted camera module in this setting. We assume that $\mathcal{S}$ securely signs the chained hash of the sequence using a private key corresponding to a publicly known identity [29].
- **Prover ($\mathcal{P}$).** An untrusted party in possession of a sequence, produced by $\mathcal{S}$, who wishes to convince the verifier of the authenticity of certain trims of the sequence, without revealing the original sequence. The prover may be fully adversarial and attempt to manipulate the revealed sequence or its accompanying proof.
- **Verifier ($\mathcal{V}$).** An entity that receives zero-knowledge proofs from $\mathcal{P}$ and checks that (i) the provided signature is authentic, (ii) the provided VC root is indeed derived from the

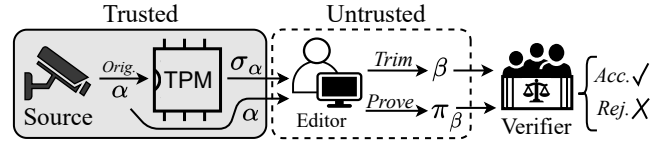


Figure 1: System model.

authentic signature, and (iii) the final provided trim belongs to the authenticated VC. Unlike in traditional centralized provenance systems, $\mathcal{V}$ does not query any third-party server; all verification is performed locally given the public keys of trusted origins.

Our setup follows prior work [13, 14, 31, 41] and industry standards such as C2PA [5].

We adopt a standard probabilistic polynomial-time (PPT) adversary model, consistent with prior work [13, 14, 41]. The adversary $\mathcal{A}$ may control the prover $\mathcal{P}$, the communication channel, or both. In particular, $\mathcal{A}$:

- Can observe, intercept, modify, delay, or inject arbitrary messages between any two parties.
- May attempt to forge VCs or ZKPs.
- May attempt to generate a ZKP for an input different from the actual underlying input.
- May try to produce multiple valid but conflicting proofs for the same video trim, or replay previously valid proofs in a different context.
- May adaptively choose inputs, queries, and proofs to maximize the probability of verifier acceptance without possessing a valid authenticated trim.

We assume $\mathcal{A}$ cannot:

- Break the unforgeability of the digital signature scheme used by $\mathcal{S}$.
- Find collisions in the cryptographic hash function used in the VC construction.
- Break the soundness or zero-knowledge properties of the underlying zkSNARK construction (e.g., Groth16 or Nova).
- Learn any information about the private signing key of the source.

4 PROVABLE CONSTRUCTION OF VECTOR COMMITMENTS

We start this section by defining a sequence hashing mechanism based on any collision-resistant block hash. Next, we demonstrate how to build a VC on top of this mechanism to achieve efficient proofs for partial reveals. Finally, we show how to trustlessly prove the construction of the VC using folding-based zkSNARKs.

4.1 Lightweight Commitments at the Source

We aim to minimize the computational overhead at the data source. A realistic assumption is that the device maintains only a sequential hash state, updated incrementally as new elements are generated [29]. At the same time, it must remain possible to commit to and later authenticate individual elements in the sequence, rather than only the dataset as a whole.

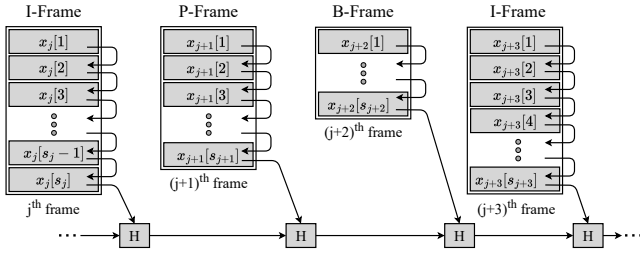


Figure 2: Data flow of the cumulative video frame hash H_Φ . This example illustrates construction of a sequence hashing method using standard block hashes. For each element x_i , we denote its size as $s_i = |x_i|$.

To achieve this, we adopt a structure based on *sequential chaining of element hashes*. This provides position binding while allowing the commitment to be updated on the fly: each new element is hashed, and the result is appended to the cumulative state. A similar approach is also suggested in prior work [29]. In many scenarios, such as video streams or network logs, element sizes may vary depending on the format. Thus, the hashing method must handle variable-sized inputs robustly and remain compatible with diverse formats.

Definition 4.1. Let s_i denote the size of the i -th element in the sequence $X = \{x_1, \dots, x_n\}$, i.e., $s_i = |x_i|$. A **format-agnostic sequence hasher** H_Φ is an algorithm that assigns a cumulative hash value to each $x_i \in X$, incorporating the hash of the previous element to maintain sequential integrity. Such a hash function can be constructed using any collision-resistant block hash function H , as illustrated in Figure 2, and is defined as follows:

$$H_\Phi : \mathbb{Z}_p^{s_i} \rightarrow \mathbb{Z}_p \mid H_\Phi(x_i) = H(\dots H(x_i[1] || x_i[2]) || \dots || x_i[s_i]) \rightarrow h_x^i$$

$$H_\Phi : \mathbb{Z}_p^{s_{i+1}} \rightarrow \mathbb{Z}_p \mid H_\Phi(h_x^{i-1}, x_i) = H(h_x^{i-1} || h_x^i) \rightarrow h_x^i$$

As illustrated in Figure 2, the cumulative hash of the sequence requires $\sum s_i$ evaluations of H . The storage cost is $O(1)$, specifically $(2 \times \log p)$ bits: one variable to store the latest cumulative state ℓ_x^i and one temporary variable to compute $H_\Phi(x_i)$ for the current element.

Maintaining only the sequential hash of the data is compact but inefficient for authenticating arbitrary disclosures such as media trims. Specifically, to authenticate a trim $\mathcal{T} = \{x_k, \dots, x_{k+m}\}$ from $X = \{x_1, \dots, x_n\}$, the prover must traverse all elements from x_k to x_n in order to reconstruct the chain from ℓ_x^k to ℓ_x^n . This inefficiency arises because there is no direct link between ℓ_x^k and ℓ_x^n without processing the entire intermediate sequence.

To address this limitation, a VC must be constructed over X so that any element can be efficiently authenticated against the root commitment [11]. Recent work [33, 38] introduces VC schemes based on optimized tree structures that offer efficient updates and proofs.

In our setting, we adopt a minimal design based on a standard Merkle tree to avoid additional complexities. We extend the tree construction with a chaining mechanism: each leaf is computed by hashing its own value together with the cumulative hash of the

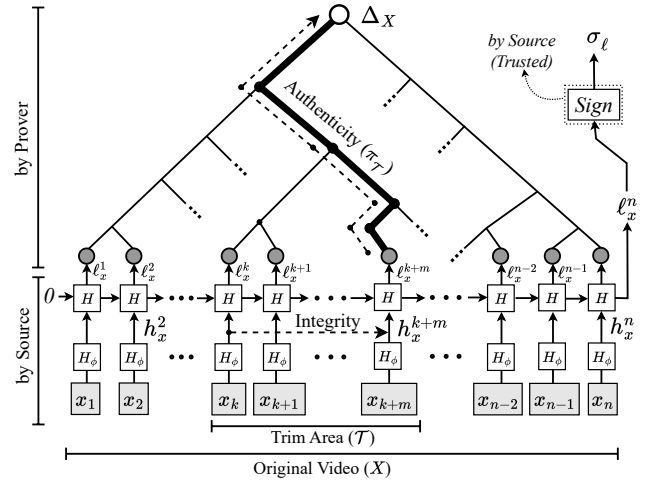


Figure 3: Proposed VC scheme for sequential data.

previous element. Such a structure can be naturally instantiated over the sequence hasher from Definition 4.1, as shown in Figure 3. This approach enables efficient trim authentication: only a single Merkle proof for the last element of the trim is required, along with the hash chain linking the start and end of the trimmed segment.

Definition 4.2. Given sequence $h_x^* = \{h_x^1, \dots, h_x^n\}$, with sequential chained hash ℓ_x^n , we construct a Merkle tree over as a quadruple PPT algorithms ($\text{KeyGen}_\Delta, \text{Com}_\Delta, \text{Open}_\Delta, \text{Vfy}_\Delta$) as follows:

- $pp \leftarrow \text{KeyGen}_\Delta(1^\lambda, n)$: The generator outputs public parameters pp based on the security parameter λ , and vector size n .
- $\Delta_X \leftarrow \text{Com}_\Delta(pp, h_x^*)$: Commits to all elements in the sequential data X according to the structure illustrated in Figure 3.
- $\delta_k \leftarrow \text{Open}_\Delta(pp, \ell_x^k, k, h_x^k, \ell_x^{k-1})$: Produces a proof (in our case, a Merkle path/proof) for opening the commitment in the k^{th} element in the sequence, where $k \in [n]$ and $\ell_x^k = H(\ell_x^{k-1} || h_x^k)$.
- $b \leftarrow \text{Vfy}_\Delta(pp, \Delta_X, \ell_x^k, k, \delta_k)$: The verifier accepts the proof δ_k if and only if $\ell_x^k = H(\ell_x^{k-1} || h_x^k)$ is indeed the k^{th} leaf of the tree with root Δ_X .

THEOREM 4.3. *The proposed Merkle tree construction defined in Definition 4.2 satisfies position-binding.*

SKETCH. Consider the commitment Δ_X and two elements, x_k and x'_k , in the sequence X . For successful verification through Vfy_Δ , the adversary $\mathcal{A}$ must be able to open both values at the same position. The probability of the adversary's success is calculated as follows:

$$\Pr \left[\begin{array}{l} \text{Vfy}_\Delta(\Delta_X, \ell_x^k, k, \delta_k), \\ \text{Vfy}_\Delta(\Delta_X, \ell_x^k, k, \hat{\delta}_k), \\ (h_x^k \neq \hat{h}_x^k \vee x_k \neq x'_k) \end{array} \mid \begin{array}{l} pp \leftarrow \text{Gen}(1^\lambda, n) \\ \Delta_X, \ell_x^k, \hat{\ell}_x^k, \delta_k, \hat{\delta}_k, x_k, x'_k \leftarrow \mathcal{A} \end{array} \right]$$

This could occur in two scenarios. First, $\mathcal{A}$ may find two preimages for the hash function H_ϕ , which is assumed to be collision-resistant under the PPT adversary model. Second, $\mathcal{A}$ could successfully open the leaf ℓ_x^k with two different values, i.e., $H(\ell_x^{k-1} \parallel h_x^k) = H(\ell_x^{k-1} \parallel \hat{h}_x^k) \wedge h_x^k \neq \hat{h}_x^k$. This would imply that the adversary has found a collision in H , which occurs with negligible probability. Consequently, the overall probability of a successful attack (above probability) is negligible, and the proposed VC satisfies *position-binding*. $\square$

4.2 Proving Construction of Δ_X from ℓ_x^n

Here, we demonstrate how to efficiently prove the trustless construction of a Merkle tree with root Δ_X based on the committed hash values of sequential data, i.e., $(\ell_x^i)_{i \in [n]}$ as follows:

Definition 4.4. Let X be the original sequence with elements $\{x_1, \dots, x_n\}$ and chained hash values $(\ell_x^i)_{i \in [n]}$, where the final hash ℓ_x^n is authenticated by a signature σ_ℓ . Let Δ_X denote the root of a Merkle tree constructed over the set $L = (\ell_x^i)_{i \in [n]}$, where each ℓ_x^i is computed as $H(\ell_x^{i-1} \parallel h_x^i)$, with $\ell_x^0 = 0$. The proof statement for constructing the Merkle tree with respect to ℓ_x^n is defined as follows.

$$\mathcal{R}_{VC} := \{((\ell_x^n, \Delta_X), ((\ell_x^i)_{i \in [n]}, (h_x^i)_{i \in [n]}, (\delta_i)_{i \in [n]})) : \ell_x^0 = 0 \wedge \forall i \in [n], \ell_x^i = H(\ell_x^{i-1} \parallel h_x^i) \wedge \forall \mathbf{y}_\Delta(\Delta_X, \ell_x^i, i, \delta_i) = 1\} \quad (1)$$

THEOREM 4.5. *The probability of a PPT adversary breaking the soundness of above statement is negligible.*

SKETCH. We refer to the sent Merkle root from adversary as Δ'_X and the signed commitment ℓ_x^n with the signature σ'_ℓ . Moreover, we assume that the sequence X consists of n elements $\{x_1, \dots, x_n\}$. Overall, we calculate the probability of the adversary $\mathcal{P}^*$ being able to generate false proofs of the statement in Definition 4.4 as follows:

$$\Pr \left[\begin{array}{l} \left(\exists i, (\forall \mathbf{y}_\Delta(\Delta_X, z'_i, i, \hat{\pi}_\Delta^i) = 1 \wedge (\delta_i \neq \hat{\delta}_i \vee z_i \neq z'_i)) \right) \\ \vee (z_i = H(z_{i-1} \parallel \hat{h}_x^i) \wedge \hat{h}_x^i \neq h_x^i) \\ \vee (z_0 = 0 \wedge z_n \neq \ell_x^n) \\ \exists i, \forall \mathbf{y}_\Delta(\Delta_X, z_i, i, \delta_i) \neq 1, \\ \forall \mathbf{y}(vk, n, z_0, z_n, \Pi) = 1 \end{array} \middle| \begin{array}{l} pp \leftarrow \mathcal{G}(1^\lambda), (pk, vk) \leftarrow \mathcal{K}(pp, s) \\ (\delta_i, z_i)_{i \in [n]}, \Pi \leftarrow \mathcal{P}^* \\ (\omega_1, \dots, \omega_n) \leftarrow \mathcal{E}(pp, z_0, z; \rho) \\ \forall i \in [n], (h_x^i, \delta_i) \leftarrow \omega_i \\ \forall i \in [n], z_i \leftarrow H(z_{i-1} \parallel h_x^i) \end{array} \right]$$

The argument proceeds in three parts: (1) If the adversary submits a valid proof with correct public parameters but finds a $\hat{h}_x^i \neq h_x^i$ such that $H(z_{i-1} \parallel \hat{h}_x^i) = H(z_{i-1} \parallel h_x^i)$, then the adversary has found a collision in H . Due to the collision resistance of H , this occurs with negligible probability. (2) Another possibility is that the adversary successfully opens a leaf in Δ_X using two different Merkle paths, $\delta_i \neq \hat{\delta}_i$, or with two distinct leaf values, $z_i \neq z'_i$. Both of these scenarios would require the adversary to find collision in H because the entire Merkle path is simply a chained application of H . (3) Alternatively, the adversary could attempt to verify a malformed proof Π' using public parameters where $z_n \neq \ell_x^n$, while ensuring $z_0 = 0$. This would require breaking the soundness property of the employed IVC scheme. Therefore, the overall probability of a PPT

adversary successfully breaking the soundness of the proof π_Δ is negligible. $\square$

To ensure efficiency, we design our proof system using a folding scheme built on the Nova protocol [25] as mentioned earlier.

5 PROTOCOL DETAILS

In this section, we present our three-phase protocol (1. *Commit*, 2. *Prove*, and 3. *Verify*) for trustlessly authenticating arbitrary trims of sequential data without revealing the original source. Figure 4 provides detailed steps that each actor in the proposed protocol takes.

Commitment Phase: During this phase, the source (e.g., a camera or a trusted application like TruePic or ProxCam [29]) commits to the captured sequential data (e.g., video) using the hash algorithm defined in Definition 4.1, as depicted in Figure 3. To finalize the commitment, the device signs the computed commitment hash ℓ_x^n and outputs both the recorded data and the corresponding digital signature σ_ℓ . We argue that this commitment process is computationally feasible on existing hardware, given that standard cryptographic hash functions such as SHA-256 are already supported in commercial recording devices [3].

Prove Phase: This phase consists of three steps: (1) The prover first proves the construction of the full VC as detailed in §4. (2) Next, the prover proves arbitrary trims of the sequence using the authenticated Δ_X . (3) Optionally, the prover may choose not to reveal the original signature and instead prove its existence to preserve the signer's identity [14].

I) Prove $\Delta_X \sim \ell_x^n$: This step is required only once per dataset. It involves constructing a Merkle tree from the recorded data outside the device using a ZK circuit. The circuit implements the statement defined in Definition 4.4, proving that the Merkle tree is correctly derived while computing the leaf hashes, as illustrated in Figure 3. The resulting proof establishes a verifiable link between Δ_X and ℓ_x^n , ensuring that Δ_X is authenticated with respect to a valid signature σ_ℓ for ℓ_x^n . This Merkle-based structure enables flexible selection of arbitrary start and end points when trimming the sequence.

II) Prove $\mathcal{T} \in X$: Once Δ_X is established, the prover generates a proof to validate the authenticity and integrity of any trimmed segment. The prover must prove the following statement.

Definition 5.1. Let X be the original sequence consisting of n elements $x_1, \dots, x_n$, and let Δ_X be the VC corresponding to ℓ_x^n . We state the verifiable trim of X for a trim $\mathcal{T}$ starting at the k -th element with a length of m as follows:

$$\mathcal{R}_{\text{trim}} := \{((\Delta_X, \ell_x^{k+m}), (\delta_{k+m})) : \forall \mathbf{y}_\Delta(\Delta_X, \ell_x^{k+m}, k+m, \delta_{k+m}) = 1\} \quad (2)$$

THEOREM 5.2. *The probability of a PPT adversary breaking the soundness of above statement is negligible.*

SKETCH. Let $\mathcal{T}' = \{x'_k, \dots, x'_{k+m}\}$ be the trimmed sequence from the adversary and Δ_X the corresponding commitment. Given the original sequence X , we aim to compute the probability that the adversary $\mathcal{P}^*$ can generate false proofs for the statement in Def. 5.1

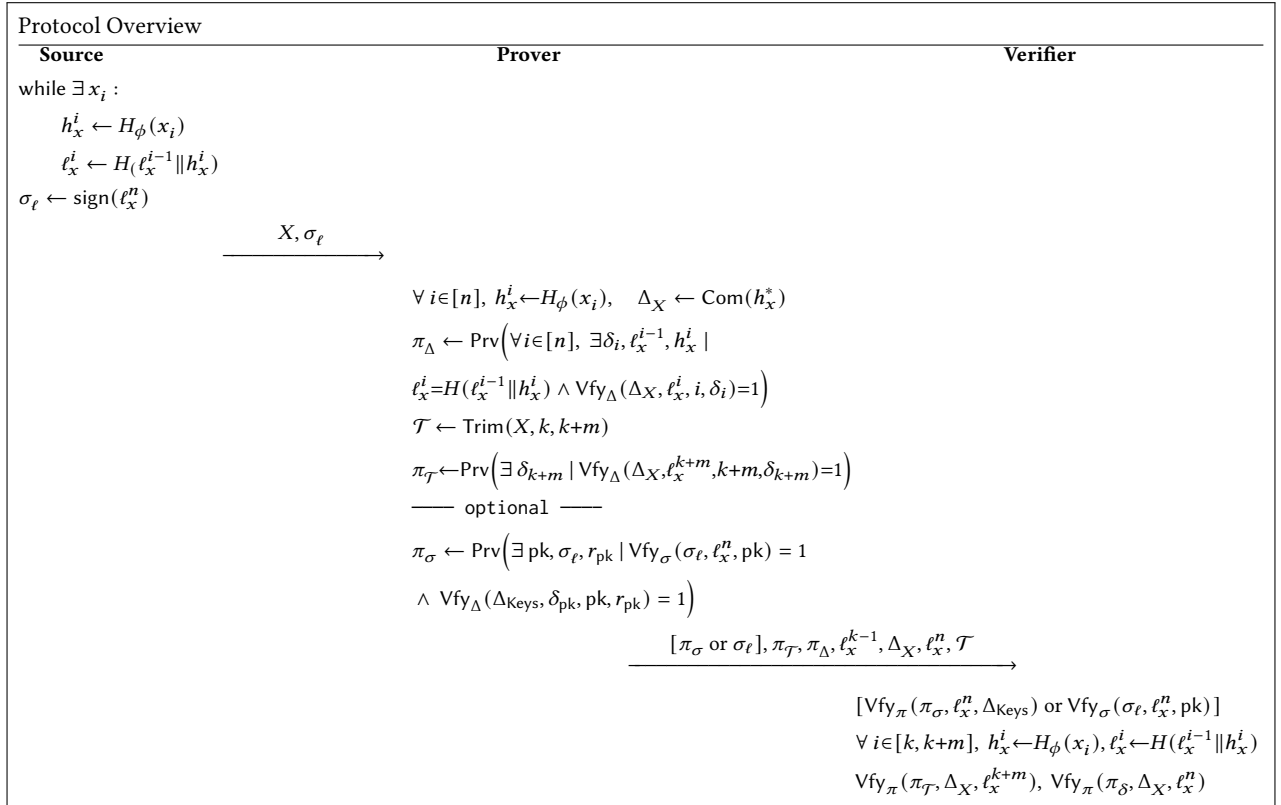
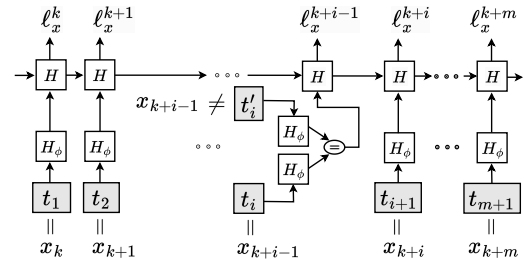


Figure 4: Overview of the protocol.

as below.

$$\Pr \left[\begin{array}{l} \text{Vfy}_\pi(vk, u, \pi_{\mathcal{T}}) = 1, \\ \text{Vfy}_\Delta(\Delta_X, \ell_x^{k+m}, k+m, \delta_{k+m}) \neq 1 \end{array} \middle| \begin{array}{l} pp \leftarrow \mathcal{G}(1^\lambda), \\ (pk, vk) \leftarrow \mathcal{K}(pp, H) \\ \mathcal{T}', \delta_{k+m}, \ell_x^{k+m-1}, h_x^{k-1} \leftarrow \mathcal{P}^* \\ \forall i \in [k, k+m] h_x^i = H_\phi(x_i) \\ \ell_x^{k+m} \leftarrow H(\ell_x^{k+m-1} || h_x^{k+m}) \end{array} \right]$$

Arguing that the above probability is negligible in the PPT model is straightforward. This follows from the collision resistance of the hash function H , H_ϕ and the soundness of the underlying zkSNARKs. Our argument proceeds in three parts: (1) Let's assume the adversary submits a valid proof with valid public parameters but manages to find an $x'_i \neq x_i$, such that $H_\phi(x'_i) = H_\phi(x_i) = h_x^i$ (Figure 5). Thus, the adversary must have found a collision in H_ϕ , which has a negligible probability due to the collision resistance property of it. (2) Another scenario is that the adversary manages to find a collision in H for calculating $H(\ell_x^{k+m-1} || h_x^{k+m})$, which is not possible in our PPT adversary model. (3) Alternatively, the adversary must manage to successfully verify a malformed proof $\pi'_{\mathcal{T}}$ using some public parameters as $\hat{\ell}_x^{k+m} \neq \ell_x^{k+m}$. The probability of this for a PPT adversary is negligible according to the soundness property of the employed zkSNARKs. Therefore, the overall probability of a PPT adversary breaking the soundness of $\pi_{\mathcal{T}}$ is negligible. $\square$

Figure 5: Breaking *position-binding* of our VC, requires finding two preimages for H_ϕ .

III) [Optional] Anonymizing Signer's Identity This optional protocol section is based on prior work [13, 14], which can be directly applied to our framework. To anonymize the original signer's identity, we assume existence of a public VC to a known set of verified public keys, established during a secure ceremony with trusted authorities. Instead of revealing the signature σ_m for the message m , the prover generates a zkSNARK proof for the following statement.

Definition 5.3. Assume that all verified public keys are committed to by a VC scheme with public value of Δ_{PK} . We define Vfy_σ such that $\text{Vfy}_\sigma(m, \sigma_m, pk) = 1$ if and only if σ_m is a valid signature for

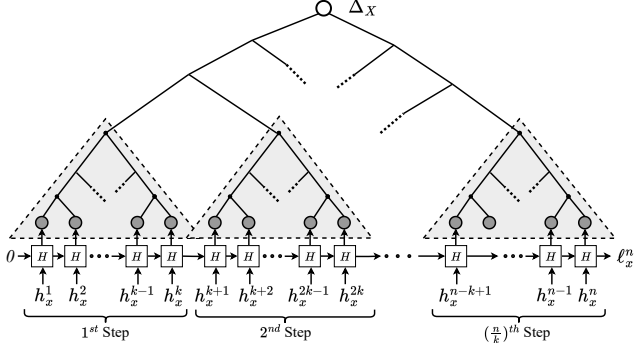


Figure 6: Proving the construction of a Merkle tree with root Δ_X , derived from a sequence of hashes in folding scheme with the final hash value ℓ_x^n . The proof has $\frac{n}{k}$ steps, each handling k leaves and computing subtrees of height $\log k$. In each step, we also assert the existence of a valid Merkle path from the root of the computed subtree to Δ_X .

m under the public key pk as follows:

$$\mathcal{R}_{\text{sig}} := \{((m, \Delta_{PK}), (\sigma_m, \delta_i, pk_i)) : \forall \text{fy}_\sigma(m, \sigma_m, pk_i) = 1 \wedge \forall \text{fy}_\Delta(\Delta_X, pk_i, i, \delta_i) = 1\} \quad (3)$$

Verification Phase: During this phase, the verifier must check at least two proofs: π_Δ and $\pi_{\mathcal{T}}$. Additionally, they need to verify either the signature σ_ℓ or a proof of its existence, π_σ . Notably, once π_Δ and either σ_ℓ or π_σ are verified, there is no need to reverify them for future trims of the same sequence.

6 IMPLEMENTATION DETAILS

In this section, we describe how the proposed protocol's proving phase can be integrated into a folding scheme to leverage the efficiency of Nova-based proofs [25].

6.1 Proving $\Delta_X \sim \ell_x^n$

Merkle tree construction, based on repeated applications of a hash function, can be viewed as a recursive process. For instance, a tree of height 4 (16 leaves) can be interpreted as a height-2 tree whose leaves are themselves roots of height-2 subtrees. This recursive structure enables the use of folding schemes, where each fold must involve identical computations. By batching hash operations to match the folding protocol's uniform structure, we optimize proof generation and significantly reduce the overhead of processing long sequences.

Figure 6 illustrates the process of proving the construction of a Merkle tree with root Δ_X from a sequence of hashes, where the final hash value corresponds to ℓ_x^n . The proof is generated within a folding-based scheme, ensuring efficient verification while maintaining the integrity of the sequential data structure.

Algorithm 1 outlines the computations performed at each folding step within our IVC setup. Specifically, we partition the Merkle tree into k subtrees (Figure 6), and prove that each subtree is part of the original tree by providing witness to the existence of a path from the root of the subtree to Δ_X . The arrays $[p_i]$ and $[s_i]$ in

the inputs represent the sibling values and their corresponding positions within the Merkle path, respectively. Additionally, in each step, we compute the cumulative hash of the subtree's leaves and propagate the final result to the subsequent step as part of the public output.

Algorithm 1: Folding Step i

Aux. In : $[h_x^{(i-1)k+1}, \dots, h_x^{i \times k}], [p_1, \dots, p_{\log(\frac{n}{k})}], [s_1, \dots, s_{\log(\frac{n}{k})}]$
Pub. In : $\Delta_X, \ell_x^{(i-1)k}$
Pub. Out : $\Delta_X, \ell_{i \times k}$

```

1 for  $j : 1 \rightarrow k$  do
2    $\ell_x^{(i-1)k+j} \leftarrow H(\ell_x^{(i-1)k+j-1} || h_x^{(i-1)k+j})$ 
3  $node \leftarrow \text{buildTree}([\ell_x^{(i-1)k+1}, \dots, \ell_x^{i \times k}])$ 
4 for  $j : 1 \rightarrow \frac{n}{k}$  do
5   if  $s_j == 0$  then
6      $node \leftarrow H(node || p_j)$ 
7   else
8      $node \leftarrow H(p_j || node)$ 
9 assert  $\Delta_X == node$ 
10 output  $\Delta_X, \ell_{i \times k}^{i \times k}$ 

```

6.2 Proving $\mathcal{T} \in X$

Algorithm 2 outlines computations necessary for this phase. As discussed in Section 5, proving the hash of the trim $\mathcal{T} = \{x_m, \dots, x_{m+k}\}$ itself is not required when the trim is provided to the verifier. This is because the verifier can independently compute the cumulative hash of the trim elements to derive h_x^{m+k} locally. Consequently, the prover only needs to show that h_x^{m+k} is a valid opening of a leaf in the Merkle tree rooted at Δ_X (see Definition 5.1). Notably, this makes the proof complexity independent of the trim size.

Algorithm 2: Proving $\mathcal{T} \in X$

Aux. In : $\ell_x^{m-1}, [p_1, \dots, p_{\log n}], [s_1, \dots, s_{\log n}]$
Pub. In : Δ_X, h_x^j

```

1  $node \leftarrow H(\ell_x^{m-1} || h_x^j)$ 
2 for  $i : 1 \rightarrow \log n$  do
3   if  $s_i == 0$  then
4      $node \leftarrow H(node || p_i)$ 
5   else
6      $node \leftarrow H(p_i || node)$ 
7 assert  $\Delta_X == node$ 

```

6.3 Verification

In this phase, the verifier must verify π_Δ and $\pi_{\mathcal{T}}$ as described in Algorithm 3. Additionally, the verifier should verify either the signature σ_ℓ or a proof of its existence, π_σ (Definition 5.3). The algorithm

begins by authenticating ℓ_x^n in line 1. Next, it verifies Δ_X with respect to the previously authenticated ℓ_x^n in line 2. Finally, the verifier authenticates the received trim $\mathcal{T}$ by computing its cumulative hash and verifying $\pi_{\mathcal{T}}$ using the already verified Δ_X and the locally computed hash value as public inputs (lines 4-6).

Algorithm 3: Verification

Pub. In : $\mathcal{T}, \Delta_X, \pi_{\Delta}, \pi_{\mathcal{T}}, \ell_x^n, (\sigma_{\ell} \text{ or } \pi_{\sigma}), \ell_x^{k-1}$
Pub. Out: *true* or *false*

```

1  $v_{\sigma} \leftarrow \text{Vfy}_{\sigma}(\ell_x^n, \sigma_{\ell}) \text{ or } \text{Vfy}_{\pi}(\ell_x^n, \pi_{\sigma})$ 
2 assert  $v_{\sigma} == 1$ 
3  $v_{\Delta} \leftarrow \text{Vfy}_{\pi}(\Delta_X, \pi_{\Delta}, \ell_x^n)$ 
4 assert  $v_{\Delta} == 1$ 
5  $temp \leftarrow \ell_x^{k-1}$ 
6 for  $i : 1 \rightarrow |\mathcal{T}|$  do
7    $temp \leftarrow H(temp || H_{\phi}(t_i))$ 
8  $v_{\tau} \leftarrow \text{Vfy}_{\pi}(\Delta_X, \pi_{\mathcal{T}}, temp)$ 
9 assert  $v_{\tau} == 1$ 

```

7 PERFORMANCE EVALUATION

In this section, we present the results related to prover time, proof size, and verification time. In the proposed scheme, the device calculates the chained hash of the data and signs the final hash value. Afterward, the prover is responsible for proving the correct construction of the proposed VC scheme (Definition 4.4) based on the chained hash values and later generating the proof for the arbitrarily trimmed data (Definition 5.1).

System Setup. Table 3 summarizes our experimental environment. To evaluate the practicality of our constructions, we benchmarked our implementation on a mid-range *Dell Latitude 5531* laptop with a 12th-generation Intel Core-i5 CPU and 16 GB of RAM running *Ubuntu 22.04 Desktop*, representing the prover's main device. The source device is a *Raspberry Pi Zero 2W* with 512 MB of RAM and a 16 GB SD card, running *Ubuntu 22.04 Server*, responsible for generating and authenticating the original sequence.

7.1 Commitment Phase

We first analyze the computational and storage complexity of the two steps in our VC construction: the sequence hash (Definition 4.1) and the tree-based structure on top of it (Definition 4.2). As discussed earlier, the commitment phase must remain lightweight to sustain high throughput and minimize delays in real-time processing. Here, we quantify the overhead of constructing a full VC on a resource-constrained device.

Table 4 summarizes approximate bitrates for different video configurations, ranging from minimal FHD 30 fps security camera footage to 4K 60 fps broadcast sources. Depending on quality and motion levels, bitrates vary from 2 Mbps to over 100 Mbps. We provide more detailed analysis of different video configurations in Appendix A. Figure 7 shows the approximate number of hashes required per second to digest the media at different bitrates, assuming 256-bit blocks. To evaluate feasibility, we benchmarked a

Table 3: Experimental Setup Configuration

Device	CPU			Mem.	SSD	OS
	Model	Freq.	Cores			
Dell Latitude 5531	Intel® Core i5 12500H	3.3~4.5 GHz	12	16 GB	512 GB	Ubuntu 22.04 LTS
Rspbry. Pi Zero 2W	Arm Cortex A53	1 GHz	4	512 MB	16 GB	Ubuntu Server 22.04 LTS

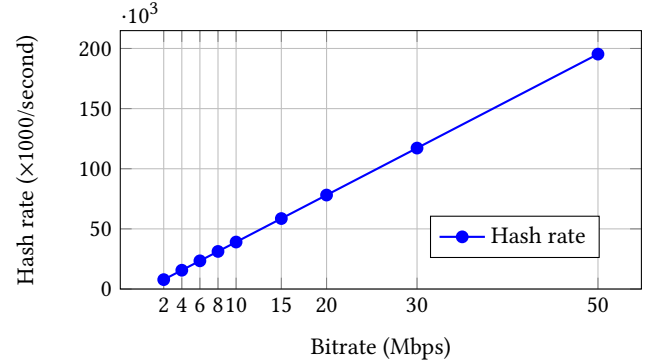


Figure 7: Hash rate required for different media bitrates.

constrained device (*Raspberry Pi Zero 2W*) against several hash algorithms, as shown in Figure 8. For standardized hash functions such as SHA and MD5, we used OpenSSL's implementations, which leverage cryptographic acceleration available on the Cortex-A53 SoC. In contrast, ZK-friendly hash functions such as Poseidon do not benefit from hardware support on such boards. The most efficient implementation we found is a C library [8], yet even for low bitrates (e.g., 2 Mbps), hashing one second of media with Poseidon exceeds one second of computation, making real-time processing infeasible. This indicates that constrained devices cannot sustain both sequence hashing and an additional VC layer when relying on Poseidon.

Table 5 compares the computational cost of the commitment phase when executed on the constrained device versus a more capable machine (e.g., a midrange laptop). On the laptop, constructing a complete VC over long footage (up to 6 hours) requires only a few minutes. On the constrained device, the same operation can exceed one hour of continuous computation. Importantly, the device must also compute H_{ϕ} over the entire footage. With Poseidon, this would take more than 10 hours for 30 minutes of footage and hundreds of hours for 6 hours of footage. By contrast, standardized hash functions with hardware acceleration on devices such as the *Raspberry Pi Zero 2W* can compute the same task in minutes (e.g., 1~5 minutes for 30 minutes of footage). This suggests that constrained devices can feasibly compute chained sequence hashes using standardized functions during capture, consistent with prior work [29].

7.2 Proving Phase

After receiving the initial commitment ℓ_x^n and its signature σ_{ℓ} from the device side, the prover constructs a Merkle tree based

Table 4: Different Video Configurations

Category	Security Camera			Dashcam			Professional (News, Live TV)		
Res. [*]	FHD	4K		FHD		4K	FHD		4K
FPS	30			30	60		50/60	24	30/60
Comp.	H.264		H.265	H.264			MPEG-2/ H.264	ProRes 422	H.265
Bitrate [★]	2 Mbps	8 Mbps	4 Mbps	10 Mbps	20 Mbps	30 Mbps	20 Mbps	220 Mbps	20 Mbps
MB/min [★]	~15 MB	~60 MB	~30 MB	~75 MB	~150 MB	~225 MB	~150 MB	~1500 MB	~150 MB
Time/GB [★]	~67 mins	~17 mins	~33 mins	~13 mins	~7 mins	~4.5 mins	~7 mins	<1 min	~7 mins

^{*} FHD: 1920×1080 and 4K: 3840×2160 ^{*} Estimated

Table 5: Commitment Complexity

	Sec. Cam		DashCam		News/TV	
Resolution	FHD	4K	FHD	4K	FHD	4K
Frame Rate	30		30	60	60	
Bitrate (Mb/S)	3	5	10	30	20	40
Hash/S in H_Φ	12 K	19 K	39 K	117 K	78 K	155 K

30 Minutes of Video	H_Φ time On R.Pi 0	MD5	1 m	1.5 m	2.4 m	7.2 m	4.8 m	10 m
		SHA2	1.3 m	1.9 m	3.2 m	9.3 m	4.6 m	12 m
		POS	1.3 h	2 h	3.3 h	10 h	6.6 h	13 h
	VC calc. by R.Pi 0	MD5	1.7 s		3.8 s			
		SHA2	2.2 s		4.8 s			
		POS	144 s		306 s			
	VC calc. by laptop	MD5	< 1 s		< 1 s			
		SHA2	< 1 s		< 1 s			
		POS	4.7 s		9.6 s			

6 Hours of Video	H_Φ time On R.Pi 0	MD5	11 m	17 m	28 m	1.5 h	1 h	2 h
		SHA2	15 m	21 m	37 m	2 h	1.5 h	2.5 h
		POS	16 h	24 h	38 h	120 h	80 h	160 h
	VC calc. by R.Pi 0	MD5	24.8 s		54.3 s			
		SHA2	33.4 s		70.3 s			
		POS	35 m		75 m			
	VC calc. by laptop	MD5	< 1 s		~2 s			
		SHA2	< 1 s		~2 s			
		POS	~1 m		~2 m			

^{*} s, m, and h in the times indicate seconds, minutes, and hours, respectively.

on the list of chained hash values and generates a proof to verify its correctness. To enhance proof generation efficiency, we employ the NOVA folding scheme [25]. Our implementation utilizes the nova-snark [6] backend and the nova-scotia front-end [7] to translate our defined steps in Circom [9] into Nova.

As illustrated in Figure 6, we optimize the Merkle tree construction by dividing it into subtrees. Table 6 presents detailed performance benchmarks for different sequence sizes. To determine the optimal subtree size proven in each step of the folding, we generated proofs for various subtree sizes while keeping the overall tree size fixed. As shown in Table 6, for a Merkle tree of height 20, increasing the subtree size reduces proving time but increases memory usage. To the best of our knowledge, this is the first benchmark for verifiable Merkle tree reconstruction within a ZKP, leveraging folding schemes.

Once the Merkle tree is constructed, the trim range can be chosen flexibly. The complexity of the proof π_T remains independent of the trim's size or position because it only proves the valid opening

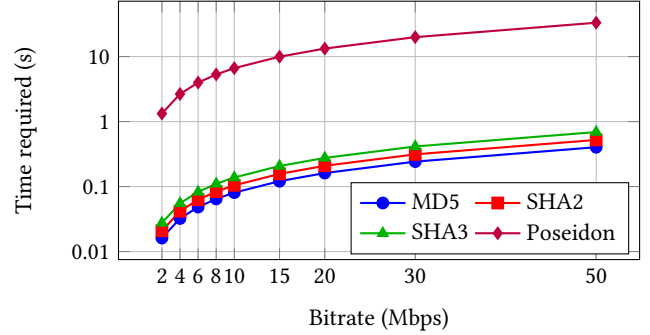


Figure 8: Log-scale comparison of time required in seconds for the source device to calculate total hashes of a 1-second video at different bitrates, across hash algorithms.

of the commitment corresponding to the last element of the trim, establishing its inclusion in our VC with a valid path to the root Δ_X . For this proof, we can employ any general zkSNARK framework. We report performance metrics for generating *Groth16* [21] proofs for this step, but alternative zkSNARKs such as Spartan [35] can also be used. A key observation is that proving the construction of the proposed VC for a long video footage (5 to 10 hours) takes less than 30 minutes on a midrange laptop, while proving multiple arbitrary trims of the sequence takes under a second. This indicates the practicality of the proposed scheme, while imposing minimal overhead to the original committer.

7.3 Verification Phase

In this phase, the verifier, who consumes the trimmed data, assesses its authenticity. As illustrated in Figure 4, the verifier checks at least two proofs (π_Δ and π_T) in addition to a signature verification or its associated anonymized proof (π_σ). Table 6 reports the proof size and verification time for both π_Δ and π_T (corresponding to the last element of the trimmed sequence) generated during the proving phase.

A key observation is the low computational overhead for the verifier and the compact proof size. For example, a trimmed video segment of several gigabytes produces a proof of only a few kilobytes, with a verification time of approximately 50 ms.

Table 6: Performance Analysis of the Proposed Constructions

video format	Commitment			$\pi_\Delta \leftarrow \mathcal{P}(\Delta_X \sim \ell_X^n)$				$\pi_{\mathcal{T}} \leftarrow \mathcal{P}(\mathcal{T} \in X)$			
				Prover		Verifier		Prover		Verifier	
	tree height	subtree height	#folds	time (m)	Mem. (GB)	time (s)	Proof size (KB)	time (s)	Mem. (GB)	time (ms)	Proof size (KB)
10 m @ 60 fps, 20 m @ 30 fps	15	7	2^8	1.15	1.9	0.17	~10	<1 s	<1	~50	~1
5 h @ 60 fps, 10 h @ 30 fps	20	12	2^8	24.7	5	2.9	~10	<1 s	<1	~50	~1
		10	2^10	26.9	3.7	0.9	~10	<1 s	<1	~50	~1
		8	2^12	34.8	3	0.3	~10	<1 s	<1	~50	~1

8 DISCUSSION AND FUTURE WORK

This paper presents a method for delegating VC construction in resource-constrained settings, targeting anonymous and privacy-preserving partial disclosures of sequential data. The main advantage of our approach is the minimal computational and storage overhead imposed on the source device, which is often constrained, while keeping the prover’s workload within practical bounds. To achieve efficient zero-knowledge proofs of full VC construction, we leverage folding-based zkSNARKs, i.e. Nova [6]. We formally analyze the soundness of our constructions under a well-defined system and adversary model.

Unlike traditional approaches, our method does not require constructing the VC at the time of commitment. Instead, it relies on computing a chained cumulative hash over the sequence elements. Any party with access to these chained hashes can later reconstruct the VC and prove the authenticity of arbitrary trims. Importantly, the complexity of verifying a trim remains constant, independent of the trim length or the underlying data complexity.

We implemented the protocol in an open-source library, available on GitHub, and evaluated its performance using a *mid-range laptop* as the *untrusted prover* and a *Raspberry Pi Zero 2W* as the resource-constrained *source device*. The results confirm the practicality of offloading VC construction in realistic scenarios. Future work will explore integrating recent advancements in folding techniques, optimizing VC schemes for even lower prover overhead and more efficient updates. Another interesting direction is to analyze the security implications of incorporating image hashing algorithms, such as *pHash*, within our framework, to potentially enhance perceptual integrity verification for visual data and decrease the computational complexity of chained hashes in the source device.

REFERENCES

- [1] 2020. Snapdragon 888 5G Mobile Platform. Accessed: 2024-10-10.
- [2] 2021. CSAM Detection. https://www.apple.com/child-safety/pdf/CSAM_Detection_Technical_Summary.pdf
- [3] 2022. Sony Unlocks In-Camera Forgery-Detection Technology. <https://www.sony.eu/presscentre/sony-unlocks-in-camera-forgery-proof-technology>. Accessed: 2024-01-08.
- [4] 2024. C2PA in DALL-E 3. <https://help.openai.com/en/articles/8912793-c2pa-in-dall-e-3>. Accessed: 2024-09-20.
- [5] 2024. C2PA Technical Specification. <https://c2pa.org/specifications/specifications/1.2/index.html>. Accessed: 2024-01-03.
- [6] 2024. Nova High-speed recursive arguments from folding schemes. <https://github.com/microsoft/Nova>. Accessed: 2024-01-03.
- [7] 2024. nova-scotia. <https://github.com/nalinbhardwaj/Nova-Scotia>. Accessed: 2024-01-03.
- [8] 2025. C and x86_64 implementation of Poseidon hash permutation. <https://github.com/CryptoExperts/poseidon>. Accessed: 2025-05-04.
- [9] Marta Bellés-Muñoz, Miguel Isabel, Jose Luis Muñoz-Tapia, Albert Rubio, and Jordi Baylina. 2022. Circom: A Circuit Description Language for Building Zero-knowledge Applications. *IEEE Transactions on Dependable and Secure Computing* (2022), 1–18. <https://doi.org/10.1109/TDSC.2022.3232813>
- [10] Dan Boneh, Benedikt Bünz, and Ben Fisch. 2019. Batching techniques for accumulators with applications to IOPs and stateless blockchains. In *Advances in Cryptology—CRYPTO 2019: 39th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18–22, 2019, Proceedings, Part I* 39. Springer, 561–586.
- [11] Dario Catalano and Dario Fiore. 2013. Vector commitments and their applications. In *Public-Key Cryptography—PKC 2013: 16th International Conference on Practice and Theory in Public-Key Cryptography, Nara, Japan, February 26–March 1, 2013. Proceedings* 16. Springer, 55–72.
- [12] Sanchuan Chen, Zhiqiang Lin, and Yinqian Zhang. 2023. Controlled data races in enclaves: Attacks and detection. In *32nd USENIX Security Symposium*. 4069–4086.
- [13] Trisha Datta, Binyi Chen, and Dan Boneh. 2024. VerITAS: Verifying Image Transformations at Scale. In *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE Computer Society, 97–97.
- [14] Stefan Dziembowski, Shahriar Ebrahimi, and Parisa Hassanizadeh. 2025. VIMz: Private proofs of image manipulation using folding-based zkSNARKs. *Proceedings on Privacy Enhancing Technologies* (2025).
- [15] Shahriar Ebrahimi and Parisa Hassanizadeh. 2024. From Interaction to Independence: zkSNARKs for Transparent and Non-Interactive Remote Attestation. In *Network and Distributed Systems Security Symposium (NDSS)*.
- [16] Jianwei Fei, Zhihua Xia, Benedetta Tondi, and Mauro Barni. 2022. Supervised gan watermarking for intellectual property protection. In *2022 IEEE WIFS. IEEE*, 1–6.
- [17] Pierre Fernandez, Guillaume Couairon, Hervé Jégou, Matthijs Douze, and Teddy Furon. 2023. The stable signature: Rooting watermarks in latent diffusion models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 22466–22477.
- [18] Pierre Fernandez, Hady Elsahar, I Zeki Yalniz, and Alexandre Mourachko. 2024. Video Seal: Open and Efficient Video Watermarking. *arXiv preprint arXiv:2412.09492* (2024).
- [19] Sergey Gorbunov, Leonid Reyzin, Hoeteck Wee, and Zhenfei Zhang. 2020. Point-proofs: Aggregating proofs for multiple vector commitments. In *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security*. 2007–2023.
- [20] Lorenzo Grassi, Dmitry Khovratovich, Christian Rechberger, Arnab Roy, and Markus Schofnegger. 2021. Poseidon: A New Hash Function for Zero-Knowledge Proof Systems. In *USENIX Security Symposium*, Vol. 2021.
- [21] Jens Groth. 2016. On the Size of Pairing-Based Non-interactive Arguments. In *Advances in Cryptology – EUROCRYPT 2016*, Marc Fischlin and Jean-Sébastien Coron (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 305–326.
- [22] Kun Hu, Zixuan Hu, Qianhui Zhu, Xiaochao Wang, and Xingjun Wang. 2024. StegaVideo: Robust High-Resolution Video Steganography with Temporal and Edge Guidance. (2024).
- [23] Patrick Jauernig, Ahmad-Reza Sadeghi, and Emmanuel Stappf. 2020. Trusted execution environments: properties, applications, and challenges. *IEEE Security & Privacy* 18, 2 (2020), 56–60.
- [24] Abhiram Kothapalli and Srinath Setty. 2022. SuperNova: Proving universal machine executions without universal circuits. *Cryptology ePrint Archive* (2022).
- [25] Abhiram Kothapalli, Srinath Setty, and Ioanna Tzialla. 2022. Nova: Recursive zero-knowledge arguments from folding schemes. In *Annual International Cryptology Conference*. Springer, 359–388.
- [26] Yuezun Li, Xin Yang, Pu Sun, Honggang Qi, and Siwei Lyu. 2020. Celeb-df: A large-scale challenging dataset for deepfake forensics. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 3207–3216.
- [27] Dongdong Lin, Benedetta Tondi, Bin Li, and Mauro Barni. 2024. A CycleGAN Watermarking Method for Ownership Verification. *IEEE Transactions on Dependable and Secure Computing* (2024).

- [28] Yuxin Liu, Yoshimichi Nakatsuka, Ardalan Amiri Sani, Sharad Agarwal, and Gene Tsudik. 2022. Vronicle: verifiable provenance for videos from mobile devices. In *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services*. 196–208.
- [29] Yuxin Liu, Zhihao Yao, Mingyi Chen, Ardalan Amiri Sani, Sharad Agarwal, and Gene Tsudik. 2024. ProvCam: A Camera Module with Self-Contained TCB for Producing Verifiable Videos. In *Proceedings of the 30th Annual International Conference on Mobile Computing and Networking*. 588–602.
- [30] Souha Mansour, Saoussen Ben Jabra, and Ezzeddine Zagrouba. 2023. A Robust Deep Learning-Based Video Watermarking Using Mosaic Generation.. In *VISIGRAPP*. 668–675.
- [31] Piotr Mikołajczyk, Parisa Hassanzadeh, and Shahriar Ebrahimi. 2025. Towards Trustless Provenance: A Privacy-Preserving Framework for On-chain Media Verification. *Cryptology ePrint Archive* (2025).
- [32] Yisroel Mirsky and Wenke Lee. 2021. The creation and detection of deepfakes: A survey. *ACM Computing Surveys (CSUR)* 54, 1 (2021), 1–41.
- [33] Charalampos Papamanthou, Shravan Srinivasan, Nicolas Gailly, Ismael Hishon-Rezaizadeh, Andrus Salumets, and Stjepan Golemac. 2024. Reckle trees: Updatable merkle batch proofs with applications. In *Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security*. 1538–1551.
- [34] Mike Schroepfer. 2019. Creating a data set and a challenge for deepfakes. *Facebook artificial intelligence* 5 (2019).
- [35] Srinath Setty. 2020. Spartan: Efficient and general-purpose zkSNARKs without trusted setup. In *Annual International Cryptology Conference*. Springer, 704–737.
- [36] Shree Singhi, Aayan Yadav, Aayush Gupta, Shahriar Ebrahimi, and Parisa Hassanzadeh. 2025. Provenance Detection for AI-Generated Images: Combining Perceptual Hashing, Homomorphic Encryption, and AI Detection Models. *arXiv preprint arXiv:2503.11195* (2025).
- [37] Lukas Struppek, Dominik Hintersdorf, Daniel Neider, and Kristian Kersting. 2022. Learning to break deep perceptual hashing: The use case neuralhash. In *Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency*. 58–69.
- [38] Ertem Nusret Tas and Dan Boneh. 2023. Vector commitments with efficient updates. *arXiv preprint arXiv:2307.04085* (2023).
- [39] Stephan Van Schaik, Alex Seto, Thomas Yurek, Adam Batori, Bader AlBassam, Daniel Genkin, Andrew Miller, Eyal Ronen, Yuval Yarom, and Christina Garman. 2024. Sok: Sgx. fail: How stuff gets exposed. In *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 4143–4162.
- [40] Junke Wang, Zuxuan Wu, Wenhao Ouyang, Xintong Han, Jingjing Chen, Yungang Jiang, and Ser-Nam Li. 2022. M2tr: Multi-modal multi-scale transformers for deepfake detection. In *Proceedings of the 2022 international conference on multimedia retrieval*. 615–623.
- [41] Chengru Zhang, Xiao Yang, David Oswald, Mark Ryan, and Philipp Jovanovic. 2024. Eva: Efficient IVC-Based Authentication of Lossy-Encoded Videos. *Cryptology ePrint Archive* (2024).

A VIDEO STANDARDS

Modern video encoding standards, such as H.264/AVC, H.265/HEVC, and VP9, employ advanced compression techniques to reduce the size of video files while maintaining acceptable visual quality. These standards achieve compression through techniques like motion estimation, temporal redundancy elimination, and spatial redundancy reduction, with a particular focus on frame-level encoding.

A typical encoded video is composed of three types of frames: *I-frames* (Intra-coded frames), *P-frames* (Predicted frames), and *B-frames* (Bidirectional predicted frames). I-frames are fully self-contained and serve as keyframes, storing all the information needed to render a complete frame without referencing other frames. In contrast, P-frames and B-frames leverage inter-frame compression by storing only the differences (or deltas) between the current frame and one or more reference frames. P-frames rely on the closest preceding I-frame or P-frame for reconstruction, while B-frames can reference both preceding and succeeding frames, enabling higher compression efficiency. This reliance on deltas introduces complexities in verifying the authenticity of individual frames or arbitrary segments since reconstructing a trimmed segment often requires access to reference frames outside the desired portion.

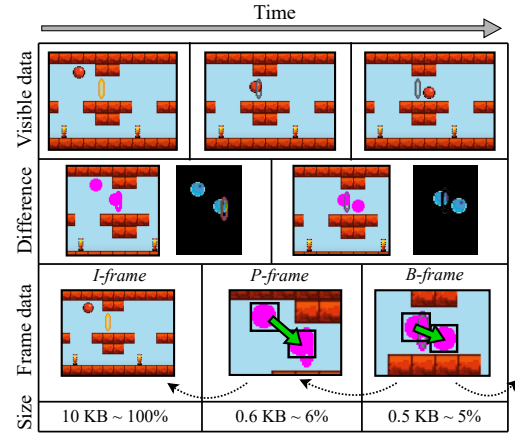


Figure 9: Example of different frame types and their impact on the lossless compression of video file size. The reported numbers are illustrative and intended to provide an intuition rather than precise data.

In archival contexts, such as CCTV and IP camera footage, additional considerations come into play. CCTV systems often employ standards like H.264 and H.265 to achieve high compression ratios, which are essential for long-term storage of continuous recordings. These systems typically use variable bit rates (VBR) to optimize storage by dynamically adjusting the compression level based on scene complexity. Furthermore, many CCTV standards adopt GOP (Group of Pictures) structures, where a sequence of frames begins with an I-frame followed by a series of P-frames and B-frames. This approach balances compression efficiency with the ability to recover from data loss or corruption in long recordings. However, the dependency on GOP structures complicates the extraction and verification of arbitrary segments, as the integrity of a segment is intrinsically tied to the availability of the corresponding I-frame and any referenced P-frames or B-frames.

IP camera standards, such as ONVIF (Open Network Video Interface Forum), extend beyond compression and storage to address interoperability and metadata integration. ONVIF-compliant devices support features like motion detection, timestamp embedding, and secure streaming protocols, which are vital for forensic analysis and legal applications. These systems also often use proprietary metadata formats to store additional information, such as camera identifiers and location data. While such metadata enhances the usability of archival footage, it raises privacy concerns and creates additional challenges for establishing trust in the authenticity of the video, especially when metadata is stripped or altered.

The diversity of video compression standards and archival practices in CCTV and IP cameras introduces unique challenges for designing commitment schemes that support selective verification. A robust approach must consider not only the compression dependencies but also the variations in metadata formats and storage practices, ensuring compatibility with industry standards while maintaining the integrity and privacy of video content.